

Yosys 形式化验证实战实验手册

从零开始验证 Verilog 模块：FIFO 与 AXI-Lite 总线

基于 SymbiYosys (SBY) 工具链

2026 年 6 月 26 日

目录

1 实验背景与目标	4
1.1 为什么 FPGA 工程师需要形式化验证？	4
1.1.1 一个真实的故事	4
1.1.2 仿真 vs 形式化验证：监控摄像头 vs 保安巡逻	4
1.1.3 形式化验证的历史脉络	5
1.2 形式化验证能做什么、不能做什么？	5
1.3 本实验目标	5
2 前置知识储备	6
2.1 形式化验证原理：从状态空间到模型检测	6
2.1.1 把 RTL 想象成一张巨大的”状态图”	6
2.1.2 为什么”暴力枚举”不可行？	7
2.1.3 符号化方法：用公式”批量”表示状态集合	7
2.1.4 Yosys 在其中的角色	7
2.2 断言：assume、assert、cover 的区别	8
2.2.1 三类原语的语义	8
2.2.2 用”考试”比喻理解三者关系	8
2.2.3 关键陷阱：assume 不是 assert	9
2.2.4 在 Verilog 中的写法	9
2.2.5 常用时序辅助函数	9
2.3 求解器：SAT/SMT 后端的工作原理	9
2.3.1 SBY 是”调度员”，求解器才是”干活的”	10
2.3.2 常用求解器对比	10

2.4	Yosys 综合基础：理解每个 Pass 在做什么	11
2.4.1	RTLIL：Yosys 的”普通话”	11
2.4.2	核心 Pass 详解	11
2.4.3	一个典型 SBY 脚本的执行流程	11
3	实验环境搭建	12
3.1	所需工具链	12
3.2	快速自检命令	12
4	实验一：同步 FIFO 状态机验证	13
4.1	实验设计思路	13
4.2	Step 1: 编写待测设计 (DUT)	13
4.2.1	设计要点：为什么用”多 1 位指针”？	15
4.2.2	Bug 在哪里？为什么仿真难发现？	15
4.3	Step 2: 编写形式化属性	15
4.3.1	为什么要这样加属性？逐条解释	18
4.4	Step 3: 编写 SBY 脚本	19
4.4.1	各段含义详解	19
4.5	Step 4: 执行验证	20
5	实验一：结果分析与调试	20
5.1	场景一：验证通过 (PASS)	21
5.2	场景二：验证失败 (Counter-example)	21
5.3	利用反例波形定位逻辑错误	22
5.4	常见调试技巧汇总	23
5.5	本实验小结	23
6	实验二：AXI-Lite 总线协议验证	24
6.1	为什么选 AXI-Lite?	24
6.2	AXI-Lite 协议要点回顾	24
6.3	Step 1: 编写 AXI-Lite Slave DUT	25
6.3.1	Bug 在哪里？	28
6.4	Step 2: 编写协议形式化属性	29
6.4.1	属性设计思路逐条解析	32
6.5	Step 3: 编写 SBY 脚本	32

6.5.1	与 FIFO 实验配置的差异	33
6.6	Step 4: 执行验证	33
6.7	结果分析: 协议违例反例	33
6.7.1	用 GTKWave 分析反例	34
6.7.2	修复方案	35
6.8	AXI-Lite 验证进阶技巧	35
6.8.1	多引擎竞赛	35
6.8.2	分阶段验证	35
6.8.3	用 [tasks] 并行多模式	36
6.9	本实验小结	36
	结语	37

1 实验背景与目标

1.1 为什么 FPGA 工程师需要形式化验证？

1.1.1 一个真实的故事

假设你刚交付了一款用于工业控制的 FPGA 设计，其中包含一个深度为 1024 的异步 FIFO，用于跨时钟域数据缓冲。仿真阶段你写了 200 个测试用例，覆盖了空读、满写、连续读写、复位恢复等各种场景，全部通过。产品上线运行 3 个月后，客户反馈：在某种罕见的”先写满、再读空、再立即写满”的操作序列下，FIFO 偶发丢数据。

你回到实验室，花了 2 周时间复现——最终发现是满标志生成逻辑在指针”回绕”时存在一个 off-by-one 错误。但你的 200 个测试用例从未触发过这种特定的指针组合。

通俗理解

这个故事的教训是：**仿真只能证明”我测过的场景没问题”，无法证明”所有场景都没问题”**。在状态空间稍大的设计中，靠人工构造测试用例实现”全覆盖”在数学上是不可能的。

1.1.2 仿真 vs 形式化验证：监控摄像头 vs 保安巡逻

理解二者区别，最直观的比喻是**监控摄像头与保安巡逻**：

- **仿真**就像在仓库里装几个监控摄像头——你只能看到摄像头对准的位置。如果小偷从盲区溜进来，摄像头拍不到，你就以为没事。每多装一个摄像头（多写一个测试用例），盲区就少一点，但永远无法保证”零盲区”。
- **形式化验证**则像派一个会”分身术”的保安，他能同时出现在仓库的每一个角落，检查每一个可能的入侵路径。如果存在任何一条能突破防线的路径，他一定会发现并告诉你”小偷会从东侧窗户的左下角翻进来，时间是凌晨 3 点 17 分”——这就是**反例**（Counter-example）。

表 1: 仿真测试与形式化验证的对比

维度	仿真测试（摄像头）	形式化验证（保安巡逻）
覆盖方式	显式构造激励，覆盖有限	数学穷举全部可达状态
Bug 发现	依赖测试用例质量，易遗漏边界	自动搜索反例，覆盖完整
反例形式	需人工分析失败波形	求解器直接给出最短反例
思考方式	”我测了什么”	”我证明了什么”
适用阶段	RTL 与门级均可	主要用于 RTL 级性质验证
计算成本	低，可并行	高，依赖求解器性能
最大短板	无法证明”没有 Bug”	状态空间过大时会”爆炸”

1.1.3 形式化验证的历史脉络

形式化验证并非新事物。它的理论基础可以追溯到 1981 年，Clarke 和 Emerson 提出的 **模型检测**（Model Checking）理论。早期它主要用于硬件验证——1992 年 Intel Pentium 浮点除法 Bug 造成了 4.75 亿美元的损失，直接推动了形式化方法在工业界的普及。如今，Intel、AMD、NVIDIA、华为海思等芯片厂商都在设计流程中深度集成形式化验证。

但长期以来，形式化验证被商业工具（如 Cadence JasperGold、Synopsys VC Formal）垄断，许可证价格高昂，普通开发者难以接触。直到 **Yosys**（由 Claire Wolf 发起的开源综合器）与 **SymbiYosys**（简称 SBY，YosysHQ 维护的形式化验证调度框架）出现，开源 EDA 才真正具备了工业级形式化验证能力。本手册正是基于这一开源工具链，让每位 FPGA 工程师都能零成本上手这一强大技术。

1.2 形式化验证能做什么、不能做什么？

在动手之前，先建立合理的期望，避免”过度神化”或”过度怀疑”。

形式化验证擅长：

- 验证**控制逻辑**的正确性：状态机、仲裁器、FIFO 指针、握手协议；
- 验证**协议合规性**：如 AXI/AHB/PCIe 的时序规则；
- 发现**边界 Bug**：off-by-one、死锁、活锁、未定义状态；
- 提供**数学证明**：在限定深度内，性质 100% 成立。

形式化验证不擅长：

- 验证**数据通路**的大规模计算：如 1024 位乘法器的数值正确性（状态空间过大）；
- 验证**模拟/混合信号**电路；
- 替代**功能仿真**：形式化验证关注”性质”，仿真关注”行为”，二者互补；
- 验证**时序收敛**：那是 STA（静态时序分析）的职责。

提示

工业界的最佳实践是”**仿真 + 形式化**”双管齐下：用仿真跑大量随机激励覆盖常见场景，用形式化验证证明关键性质在所有场景下成立。两者结合，才能达到流片级置信度。

1.3 本实验目标

本手册包含**两个递进的实战实验**：

1. 实验一：同步 FIFO 状态机验证（第 4-5 章）

以一个深度为 4 的带握手同步 FIFO 为 DUT，从零编写形式化属性，演示”验证失败 →

反例分析 → 修复缺陷”的完整调试闭环。这是入门形式化验证的最佳案例——状态空间小、逻辑典型、Bug 容易埋设。

2. 实验二：AXI-Lite 总线协议验证（第 6 章）

以一个简化的 AXI-Lite Slave 为 DUT，验证其是否符合 ARM AMBA 协议规范。这是形式化验证在工业界最典型的应用场景——协议合规性检查。我们将演示如何把协议文档中的文字规则翻译成 `assume/assert`。

完成本手册后，读者应具备以下能力：

- 理解形式化验证的工作原理，能向同事讲清“它为什么能发现仿真找不到的 Bug”；
- 掌握 Yosys 与 SBY 工具链的基本使用，能独立编写 `.sby` 配置；
- 能为任意 Verilog 模块编写 `assume/assert/cover` 三类形式化属性；
- 能解读 SBY 日志，区分“通过/失败/超时”，并利用反例波形定位缺陷；
- 能将协议文档中的时序规则翻译成形式化断言，完成协议合规性验证。

2 前置知识储备

本章是后续实验的理论地基。我们会用大量比喻和图示把抽象概念讲透——理解到位后，编写属性时才不会“知其然而不知其所以然”。

2.1 形式化验证原理：从状态空间到模型检测

2.1.1 把 RTL 想象成一张巨大的“状态图”

任何同步数字电路，本质上都是一个**有限状态机**（FSM）。我们可以这样建模：

- **节点（状态）**：所有寄存器与存储元件的当前取值组合。例如一个设计有 10 个触发器，那么理论上有 $2^{10} = 1024$ 个状态；
- **边（状态转移）**：给定当前状态 + 输入，下一拍的状态是唯一确定的（由组合逻辑计算得出）；
- **初始节点**：复位后的状态，通常是所有寄存器归零。

通俗理解

通俗比喻：想象一张地铁线路图——每个车站是一个“状态”，每条铁轨是一条“转移边”。复位状态是“始发站”。列车从始发站出发，沿着铁轨开往各个车站。形式化验证要回答的问题是：“是否存在一条路线，能让列车开到一个‘违例站’？”如果存在，把这条路线画出来就是反例；如果不存在，就证明设计安全。

2.1.2 为什么”暴力枚举”不可行？

一个看似不大的设计，状态数会爆炸性增长。例如：

- 10 个触发器： $2^{10} = 1,024$ 个状态——可以暴力枚举；
- 30 个触发器： $2^{30} \approx 10$ 亿个状态——勉强能枚举；
- 60 个触发器： $2^{60} \approx 10^{18}$ 个状态——即使每秒检查 10 亿个，也要 31 年；
- 一个典型 SoC 可能有上千个触发器——状态数超过宇宙中的原子数。

这就是著名的**状态爆炸问题**（State Explosion Problem）。直接枚举每一个状态是不可行的。

2.1.3 符号化方法：用公式”批量”表示状态集合

现代求解器采用**符号化方法**（Symbolic Method）绕开状态爆炸。核心思想是：不再逐个枚举状态，而是用一个数学公式表示”一整批状态”。

- **BDD**（Binary Decision Diagram）：用一种紧凑的图结构表示布尔函数，可以”批量”操作状态集合；
- **SAT**（Boolean Satisfiability）：把”是否存在违例”转化为一个布尔可满足性问题——”是否存在一组输入，使得某个逻辑表达式为真？”；
- **SMT**（Satisfiability Modulo Theories）：SAT 的增强版，支持位向量、数组、整数等更丰富的数据类型，更贴近硬件描述。

通俗理解

通俗比喻：暴力枚举像”挨家挨户敲门问有没有人在家”；符号化方法像”在小区门口贴一张告示：’家里有人请举手’，然后一次性统计”。前者要跑 1000 趟，后者只要 1 趟。

2.1.4 Yosys 在其中的角色

Yosys 是一座”翻译桥梁”——它把人类写的 Verilog RTL 翻译成求解器能消化的形式：

1. `read_verilog`：读入 Verilog 源码，构建内部 RTLIL 中间表示；
2. `proc / opt`：把行为级语句翻译成门级网表（AND/OR/NOT/DFE）；
3. `prep`：形式化专用预处理，把 `assume/assert` 转换为特殊的”性质检查单元”；
4. 把网表 + 性质交给后端求解器（Z3/Boolector），求解器判定”是否存在违例轨迹”。

当我们在设计中加入 `assert(P)` 时，求解器实际求解的是这样一个问题：

”是否存在一条从复位出发的轨迹，使得 $\neg P$ 在某一拍成立？”

若存在 $\rightarrow$ 返回这条轨迹（反例）；若不存在（在限定深度内） $\rightarrow$ 报告通过。这就是 `sby --mode prove` 背后的数学原理。

2.2 断言：assume、assert、cover 的区别

2.2.1 三类原语的语义

在 Verilog 中（更准确地说是在 SystemVerilog Assertion 的子集，Yosys 通过 `read_verilog -formal` 支持），有三类性质描述原语。它们的语义截然不同，初学者极易混淆——**混淆它们是形式化验证最常见的“坑”**。

表 2: 三类形式化原语的语义对比

原语	语义	典型用途
assume	约束 求解器：只考虑满足该条件的输入序列。相当于“外部环境承诺如此”	限定复位序列、约束输入握手协议、限制输入范围
assert	要求 求解器证明：在所有满足 assume 的输入下，该条件恒成立	验证设计的功能正确性、不变式、协议合规性
cover	检查 求解器：是否存在至少一条轨迹能触发该条件	评估验证完备性，确认边界场景被覆盖

2.2.2 用“考试”比喻理解三者关系

通俗理解

把形式化验证想象成一场考试：

- **assume = 考试范围**：“只考第 1-5 章”。它缩小了求解器的搜索范围。如果学生（设计）在第 6 章出错，老师（求解器）不会管——因为那不在考试范围内。
- **assert = 评分标准**：“第 3 章的定理必须证明正确”。这是要验证的目标。如果学生在范围内答错，老师会标红并指出错在哪一步（反例）。
- **cover = 知识点覆盖检查**：“确保试卷里至少有一道题考到第 4 章的难点”。如果 cover 失败，说明试卷出得太简单，没覆盖到关键场景——验证本身不够完备。

2.2.3 关键陷阱：assume 不是 assert

注意

最常见的错误：把本该 assert 的性质写成 assume。

- `assert(!full || !wr_en)`: 要求求解器证明”满时不能写”——如果设计有 Bug 导致满时仍允许写，会失败；
- `assume(!full || !wr_en)`: 告诉求解器”外部不会满时写”——求解器会相信这句话，跳过所有违例轨迹，给出虚假的”通过”。

前者是验证设计，后者是”自欺欺人”。方向一定要分清：`assume` 是”我相信外部会这样”，`assert` 是”我保证内部会这样”。

2.2.4 在 Verilog 中的写法

这三者可以直接以系统任务的形式出现在 `always` 块或 `initial` 块中：

```
1 // 假设：复位信号在第 0 拍必须为 0 (强制初始复位)
2 initial assume(!rst_n);
3
4 // 假设：满时外部不会写 (约束输入握手)
5 always @(posedge clk) if (full) assume(!wr_en);
6
7 // 断言：满与空不可能同时成立 (不变式)
8 always @(posedge clk) if (rst_n) assert(!(full && empty));
9
10 // 覆盖：希望至少看到一次”满”状态被触发
11 always @(posedge clk) cover(full);
```

Listing 1: 形式化原语的基本写法

2.2.5 常用时序辅助函数

Yosys 形式化模式支持一组 SystemVerilog 时序函数，让属性编写更简洁：

提示

`$past` 在第 0 拍返回 X (未知)，直接用它做断言会触发”伪反例”。正确做法是用 `$initstate` 守卫：`if (!$initstate) assert($past(x) == ...)`。

2.3 求解器：SAT/SMT 后端的工作原理

表 3: 常用形式化时序函数

函数	语义
\$past(x)	上一拍 x 的值 (第 0 拍返回 X)
\$past(x, n)	前 n 拍 x 的值
\$rose(x)	x 从 0 变 1 (本拍为 1 且上拍为 0)
\$fell(x)	x 从 1 变 0
\$stable(x)	x 本拍与上拍相同
\$changed(x)	x 本拍与上拍不同
\$initstate	第 0 拍 (复位拍) 为真, 其余为假
\$onehot(x)	x 中恰好有 1 位为 1
\$onehot0(x)	x 中最多 1 位为 1

2.3.1 SBY 是”调度员”，求解器才是”干活的”

SymbiYosys 本身只是一个**调度框架**——它负责读配置、调 Yosys 综合网表、启动求解器、收集结果。真正”干活”的是后端的**求解器** (Solver)。

工作流程如下：

1. SBY 调用 Yosys, 把 RTL 综合成门级网表 (AND/OR/NOT/DFE);
2. Yosys 把”是否存在违例”转化为一个 SAT/SMT 公式;
3. SBY 把公式丢给求解器 (如 Z3);
4. 求解器判定公式是否可满足:
 - **可满足 (SAT)**: 存在违例 → 求解器返回一组让公式成立的输入取值, 这就是反例;
 - **不可满足 (UNSAT)**: 不存在违例 → 性质成立, 验证通过。
5. SBY 汇总结果, 必要时把反例转成 VCD 波形。

2.3.2 常用求解器对比

表 4: SBY 常用求解引擎对比

引擎	特点	适用场景
smtbmc z3	Yosys 内置 BMC + Z3 SMT 后端, 支持位向量/数组	通用, 入门首选
smtbmc boolector	同上但用 Boolector, 位向量求解更快	大型位向量设计
smtbmc bitwuzla	Boolector 后继, 性能更优	同上, 新版推荐
aigbmc	基于 AIG 的纯 SAT BMC, 无 SMT 开销	纯布尔控制逻辑
pdr	基于属性可达性分析, 可证明无时限不变式	prove 模式
abc	综合优化引擎, 辅助其他引擎	组合逻辑等价检查

通俗理解

通俗比喻：SBY 像一个工程总监，手下有多个工种（求解器）。遇到一个验证任务，他可以同时派 Z3、Boolector、PDR 三个工种去干，谁先干完就采用谁的结果——这就是 SBY 的“多引擎竞赛”机制，是它提升效率的关键设计。

2.4 Yosys 综合基础：理解每个 Pass 在做什么

Yosys 是一个模块化的综合框架，其内部由一系列 **Pass**（处理步骤）组成。理解几个最基础的 Pass 对调试 .sby 脚本至关重要——.sby 的 [script] 段本质上就是一串 Yosys Pass 命令。

2.4.1 RTLIL：Yosys 的“普通话”

Yosys 内部用一种叫 **RTLIL**（RTL Intermediate Language）的中间表示统一描述所有设计。无论你用 Verilog、VHDL 还是 BLIF 输入，Yosys 第一步都是翻译成 RTLIL。后续所有 Pass 都在 RTLIL 上操作。这种“统一中间表示”的设计让 Yosys 极具扩展性。

2.4.2 核心 Pass 详解

read_verilog -formal 读入 Verilog 源码并构建 RTLIL。-formal 选项会启用对 `assume / assert / cover / $past` 等形式化原语的支持——不加这个选项，Yosys 会把这些原语当作普通系统任务，报“未定义”错误。

hierarchy 展开**模块实例化**层次，把顶层模块引用的子模块全部展开到同一张网表中。同时检查端口连接是否匹配。这一步是后续所有优化的前提——不展开层次，优化器看不到子模块内部，无法做跨模块优化。

proc 把 `always` 块中的过程式语句（`if`、`case` 等）转换为多路选择器与触发器的组合逻辑。Verilog 的“行为级”描述在这一步被翻译为“结构级”网表。可以理解为：把“程序员思维”的代码翻译成“电路思维”的连接图。

opt 通用优化 Pass，执行常量传播、死代码消除、冗余逻辑合并等。通常会反复调用多次以获得更紧凑的网表，从而降低求解器负担——网表越小，求解器越快。

memory 把 Verilog 中的 `reg [...]` `mem [...]` 数组转换为触发器阵列或 Block RAM。形式化验证中通常转换为触发器（便于求解器处理），这也是为什么大数组会让验证变慢的原因。

prep 形式化验证专用的“预处理”流程，内部依次调用 `hierarchy` → `proc` → `opt` → `memory` → `opt` → `techmap` → `opt`，是 SBY 默认使用的综合脚本骨架。绝大多数情况下，`prep -top <module>` 就够了。

2.4.3 一个典型 SBY 脚本的执行流程

```

1 1. SBY 读取 .sby 配置
2 2. 创建工作目录 fifo/src/
3 3. 复制 [files] 列出的源文件到工作目录
4 4. 调用 Yosys 执行 [script] 段:
5     read_verilog -formal ... -> 读入 RTL, 构建 RTLIL
6     hierarchy -top ...      -> 展开模块层次
7     prep -top ...           -> 形式化预处理 (proc/opt/memory/...)
8 5. Yosys 输出综合后的网表 (.json 或 .smt2 格式)
9 6. SBY 启动 [engines] 段的求解器, 喂入网表
10 7. 求解器逐拍展开, 检查 assert 是否被违反
11 8. 若违例: 求解器返回反例, SBY 转成 VCD
12 9. SBY 汇总结果, 打印 summary

```

Listing 2: SBY 内部执行流程 (简化版)

理解这个流程后, 再阅读 SBY 自动生成的日志就不会感到陌生——每一行日志都对应流程中的某一步。

3 实验环境搭建

由于环境已搭建完毕, 本节仅简要列出依赖关系, 便于读者在其它机器上复现。

3.1 所需工具链

表 5: 工具链组件清单

组件	版本建议	作用
Yosys	≥ 0.20	RTL 综合与形式化前端
SymbiYosys (sby)	≥ 0.45	形式化验证调度框架
Z3	≥ 4.11	SMT 求解器 (可选但推荐)
Boolector / Bitwuzla	任意近期版本	位向量求解器 (可选)
GTKWave	任意版本	查看反例 VCD 波形

3.2 快速自检命令

打开终端, 依次执行以下命令, 确认每条都能正常输出版本号:

```

1 yosys -V          # 应输出 yosys 0.x+xx
2 sby -V            # 应输出 sby 0.x
3 z3 --version      # 应输出 Z3 [version]
4 which gtkwave     # 应返回 gtkwave 的路径

```

Listing 3: 环境自检命令

若某条命令报”command not found”，请回到对应项目的安装文档补齐依赖。本实验后续步骤默认以上工具均已就绪。

提示

建议为本次实验单独建立一个工作目录，例如 `mkdir -p ~/fifo_formal && cd ~/fifo_formal`，所有 `.v` 与 `.sby` 文件都放在该目录下，便于管理。

4 实验一：同步 FIFO 状态机验证

4.1 实验设计思路

在动手写代码前，先想清楚”我们要验证什么”。这是形式化验证与仿真的根本区别——仿真往往”先写设计再补测试”，形式化验证则要求”先想清楚性质，再写属性”。

思考一下

一个深度为 4 的同步 FIFO，从用户视角看，应该满足哪些”不变式”？

1. 元素个数永远在 $[0, 4]$ 之间——不会出现负数或超过容量；
2. `empty` 为真当且仅当元素个数为 0；
3. `full` 为真当且仅当元素个数为 4；
4. `empty` 与 `full` 不能同时为真；
5. 满时不应再写、空时不应再读（这是”合理使用”的约束）。

这些不变式就是我们要写成 `assert` 的内容。把它们想清楚，属性代码自然就出来了。

4.2 Step 1: 编写待测设计 (DUT)

下面是一个深度为 4、数据宽度 8 位的同步 FIFO。我们故意在 `full` 标志的逻辑中埋设一个 Bug——这个 Bug 在常规仿真中很难被发现，但形式化验证能瞬间抓出来。

```
1 // 同步 FIFO: 深度 4, 宽度 8 位
2 // 采用"多 1 位指针"的经典设计: wr_ptr / rd_ptr 均为 PTR_W+1 位
3 module fifo #(
4     parameter DEPTH = 4,           // 必须为 2 的幂
5     parameter WIDTH = 8
6 )(
7     input  wire      clk,
8     input  wire      rst_n, // 低有效异步复位
9     // 写端口
10    input  wire      wr_en,
11    input  wire [WIDTH-1:0] wr_data,
12    output wire      full,
```

```

13 // 读端口
14 input wire rd_en,
15 output wire [WIDTH-1:0] rd_data,
16 output wire empty
17 );
18 // 指针位宽:  $\log_2(DEPTH) + 1$  (多 1 位用于区分满/空)
19 localparam PTR_W = $clog2(DEPTH);
20
21 reg [PTR_W:0] wr_ptr; // 注意: 是 PTR_W:0, 共 PTR_W+1 位
22 reg [PTR_W:0] rd_ptr;
23 reg [WIDTH-1:0] mem [0:DEPTH-1];
24
25 // 写操作
26 always @(posedge clk or negedge rst_n) begin
27     if (!rst_n) begin
28         wr_ptr <= 0;
29     end else if (wr_en && !full) begin
30         mem[wr_ptr[PTR_W-1:0]] <= wr_data;
31         wr_ptr <= wr_ptr + 1'b1;
32     end
33 end
34
35 // 读操作
36 always @(posedge clk or negedge rst_n) begin
37     if (!rst_n) begin
38         rd_ptr <= 0;
39     end else if (rd_en && !empty) begin
40         rd_ptr <= rd_ptr + 1'b1;
41     end
42 end
43
44 // 读数据组合输出
45 assign rd_data = mem[rd_ptr[PTR_W-1:0]];
46
47 // 空标志: 两指针完全相等
48 assign empty = (wr_ptr == rd_ptr);
49
50 // 满标志: 故意写错 —— 用低 PTR_W 位比较, 导致"半满"即误判满
51 assign full = (wr_ptr[PTR_W-1:0] == rd_ptr[PTR_W-1:0]); // BUG!
52
53 // 形式化属性将在 Step 2 中通过包装模块加入, 保持 DUT 干净
54 endmodule

```

Listing 4: 文件名: fifo.v ——待测的同步 FIFO (含故意 Bug)

4.2.1 设计要点：为什么用”多 1 位指针”？

这是 FIFO 设计的经典技巧，值得展开讲：

- 如果只用 $\log_2(\text{DEPTH})$ 位指针，那么”满”和”空”的条件都是 `wr_ptr == rd_ptr`——无法区分；
- 多用 1 位后，”空”是两指针完全相等（包括最高位），”满”是最高位不同但低位相等。这样空和满就有了不同的判别条件。

通俗理解

通俗比喻：想象一个圆形跑道，4 个跑道位（0/1/2/3）。两辆车（写指针与读指针）同向行驶。”空”是两车在同一位置；”满”是两车正好差一整圈——位置看起来一样，但实际跑的圈数不同。多出来的那 1 位指针就是”圈数计数器”。

4.2.2 Bug 在哪里？为什么仿真难发现？

第 41 行的 `full` 逻辑只比较了低 `PTR_W` 位（即低 2 位），忽略了最高位（圈数位）。这意味着：只要 `wr_ptr` 与 `rd_ptr` 的低 2 位相等，`full` 就会被置 1。

- 复位后 `wr_ptr=00`, `rd_ptr=00`：低 2 位相等 $\rightarrow$ `full=1`！但此时 `empty` 也为 1，明显矛盾；
- 写入 1 个数据后 `wr_ptr=01`, `rd_ptr=00`：低 2 位不等 $\rightarrow$ `full=0`；
- 写入 2 个数据后 `wr_ptr=10`, `rd_ptr=00`：低 2 位不等 $\rightarrow$ `full=0`；
- 写入 4 个数据后 `wr_ptr=00`, `rd_ptr=00`（回绕）：低 2 位相等 $\rightarrow$ `full=1`，正确；
- 但写入 2 个、读出 1 个、再写入 1 个时，`wr_ptr=10`, `rd_ptr=01`，低 2 位不等 $\rightarrow$ `full=0`，但实际只有 2 个元素，正确；
- **真正的 Bug 场景：**写入 2 个、读出 2 个（指针回到 `wr=10`, `rd=10`），此时低 2 位相等 $\rightarrow$ `full=1`，但 FIFO 实际是空的！

提示

这个 Bug 在仿真中极易被遗漏——只要测试用例不触发”写满一半再读空”的特定序列，就观察不到错误。而形式化验证会自动搜索所有可能的输入序列，瞬间找到这个反例。

4.3 Step 2: 编写形式化属性

形式化属性可以写在 DUT 内部，也可以通过**包装模块**挂接到 DUT 上。后者能保持 DUT 代码干净，是工程上的推荐做法。我们新建一个 `fifo_formal.v` 文件，在其中定义包装模块，实例化 `fifo` 并在其周围添加性质。

```

1  `ifndef FORMAL
2  module fifo_formal;
3      // 参数与 DUT 保持一致
4      localparam DEPTH = 4;
5      localparam WIDTH = 8;
6
7      reg          clk = 0;
8      reg          rst_n = 0;
9      reg          wr_en = 0;
10     reg  [WIDTH-1:0] wr_data = 0;
11     reg          rd_en = 0;
12     wire          full;
13     wire  [WIDTH-1:0] rd_data;
14     wire          empty;
15
16     // 实例化待测设计
17     fifo #(.DEPTH(DEPTH), .WIDTH(WIDTH)) dut (
18         .clk(clk), .rst_n(rst_n),
19         .wr_en(wr_en), .wr_data(wr_data), .full(full),
20         .rd_en(rd_en), .rd_data(rd_data), .empty(empty)
21     );
22
23     // ----- 时钟与复位约束 -----
24     always @(posedge clk) begin
25         // 假设 1: 复位信号在最初 2 拍保持有效, 之后释放
26         // 这样求解器从一个确定的复位状态开始探索
27         assume(!$rose(rst_n) || $past(rst_n));
28         // 假设 2: 复位释放后不再回到 0 (避免中途复位引入复杂场景)
29         if ($past(rst_n)) assume(rst_n);
30     end
31
32     initial begin
33         assume(!rst_n);    // 第 0 拍强制处于复位
34     end
35
36     // ----- 输入约束 -----
37     always @(posedge clk) begin
38         // 假设 3: 满时不能写、空时不能读 (模拟 "合理" 的外部握手)
39         if (full)  assume(!wr_en);
40         if (empty) assume(!rd_en);
41         // 假设 4: wr_data 取值范围限制 (本例无实际约束, 仅演示写法)
42         // assume(wr_data < 100);
43     end
44
45     // ----- 功能断言 -----

```



```

46     always @(posedge clk) begin
47         if (rst_n) begin
48             // 断言 1: 满与空互斥
49             assert(!(full && empty));
50         end
51     end
52
53     // ----- 影子计数器: 独立跟踪 FIFO 中元素个数 -----
54     reg [31:0] count;
55     always @(posedge clk or negedge rst_n) begin
56         if (!rst_n) begin
57             count <= 0;
58         end else begin
59             case ({wr_en && !full, rd_en && !empty})
60                 2'b10: count <= count + 1; // 只写
61                 2'b01: count <= count - 1; // 只读
62                 default: count <= count; // 都写都读 / 都不操作
63             endcase
64         end
65     end
66
67     always @(posedge clk) begin
68         if (rst_n) begin
69             // 断言 4: count 始终在合法范围
70             assert(count <= DEPTH);
71             // 断言 5: count == 0 当且仅当 empty
72             assert((count == 0) == empty);
73             // 断言 6: count == DEPTH 当且仅当 full
74             assert((count == DEPTH) == full); // 这里会触发反例!
75         end
76     end
77
78     // ----- 覆盖目标 -----
79     always @(posedge clk) begin
80         cover(full); // 至少看到一次满
81         cover(empty); // 至少看到一次空
82         cover(count == 2); // 至少看到一次半满
83     end
84
85     // 生成时钟
86     initial begin
87         #5 clk = 1;
88         forever #5 clk = ~clk;
89     end
90 endmodule
91 `endif

```

4.3.1 为什么要这样加属性？逐条解释

这是本实验最关键的部分，逐条解释每条 `assume/assert` 背后的思考过程：

假设 1 & 2（复位约束） 形式化验证必须从一个确定的初始状态开始，否则求解器会从任意状态出发，导致大量“伪反例”——这些反例并非设计 Bug，而是求解器从“未定义状态”出发导致的。我们强制第 0 拍处于复位、且复位释放后不再返回，是为了让搜索空间收敛到“正常工作场景”。

为什么不能让求解器自由探索复位？ 因为很多设计的“未复位状态”是 `X`（未知），求解器会把 `X` 当作“任意值”，从而构造出永远不可能在真实硬件上出现的轨迹。这些轨迹违例了断言，但与设计正确性无关——纯属噪声。

假设 3（握手约束） FIFO 的 `full / empty` 是建议性标志，外部理论上可以无视。但若约束，求解器会构造“满时仍强行写”的激励，触发未定义行为（`mem` 被越界写、`wr_ptr` 继续自增），产生大量与功能无关的反例。我们用 `assume` 限定外部遵守握手，专注于验证“在合法使用下设计是否正确”。

关键判断：握手是“外部协议”还是“设计保证”？如果是协议（外部承诺），用 `assume`；如果是设计必须保证的（如 `full` 时设计内部不会自增 `wr_ptr`），用 `assert`。本例属于前者。

影子计数器（count） 这是形式化验证中非常实用的技巧——用一个独立的、逻辑简单的辅助寄存器跟踪某个量，再用 `assert` 把它与 DUT 的标志位绑定。

为什么不直接断言 full 的逻辑？ 因为 `full` 的逻辑本身就是我们要验证的对象——直接断言它等于某个表达式，等于“用自己证明自己”，没有意义。影子计数器是独立实现的（用 `wr_en/rd_en` 简单加减），它与 DUT 的 `full/empty` 是两套独立逻辑。若 DUT 的 `full` 有 Bug，`count` 与 `full` 必然出现不一致，求解器会立刻抓到。

断言 6 是“陷阱” 由于我们在 Step 1 故意写错了 `full` 逻辑，断言 6 (`count == DEPTH`) == `full` 必然失败。这正是我们用来演示“反例调试”的入口。修复 DUT 后，这条断言应该能通过。

cover 目标 用于评估验证完备性。如果 `cover(full)` 失败，说明求解器在限定深度内从未让 FIFO 写满，可能意味着约束过严或验证深度不足。`cover` 失败不代表设计有 Bug，但代表“验证本身不够充分”。

注意

`assume` 与 `assert` 的方向性一定要分清：`assume` 是“我相信外部会这样”，`assert` 是“我保证内部会这样”。如果把本该 `assert` 的性质写成 `assume`，相当于“自欺欺人”——求解器会跳过违例，给出虚假的“通过”结果。

4.4 Step 3: 编写 SBY 脚本

SymbiYosys 使用 INI 风格的 `.sby` 文件作为配置入口。一个 `.sby` 文件通常包含 `[tasks]`、`[options]`、`[engines]`、`[script]`、`[files]` 五个段。下面是本实验使用的完整配置，请保存为 `fifo.sby`：

```
1 # =====
2 # [options] 段：全局验证模式与参数
3 # =====
4 [options]
5 mode prove          # 验证模式：prove（证明不变式） /
    bmc（有界模型检测） / cover
6 depth 20            # 搜索深度：从复位起最多展开 20 拍
7 multiclock off      # 本设计只有单时钟，关闭多时钟模式
8 wait 0              # 不等待用户输入，直接开始
9
10 # =====
11 # [engines] 段：指定后端求解器（可多行，SBY 会并行调度）
12 # =====
13 [engines]
14 smtbmc z3           # 主引擎：Yosys 内置 SMTBMC + Z3 求解器
15 # smtbmc boolector  # 备选引擎（注释掉，需要时启用）
16
17 # =====
18 # [script] 段：Yosys 综合脚本，决定如何把 RTL 编译成网表
19 # =====
20 [script]
21 read_verilog -formal -D FORMAL fifo.v fifo_formal.v
22 hierarchy -top fifo_formal
23 prep -top fifo_formal
24 chformal -assume -early
25
26 # =====
27 # [files] 段：列出参与编译的源文件
28 # =====
29 [files]
30 fifo.v
31 fifo_formal.v
```

Listing 6: 文件名：fifo.sby —— SymbiYosys 配置文件

4.4.1 各段含义详解

`[options]` 控制 SBY 的整体行为。

- `mode prove`：要证明性质在所有可达状态下成立（无时限），最严格但最慢；

- `mode bmc`: 只检查有限深度内是否有反例，速度更快但不完备——深度外的 Bug 抓不到；
- `mode cover`: 只检查 `cover` 目标是否可达；
- `depth 20`: 限定搜索深度，避免证明模式下某些性质长时间不收敛。

[engines] 指定一个或多个求解引擎。`smtbmc z3` 是最常用的组合：`smtbmc` 是 Yosys 内置的有界模型检测器，`z3` 是其调用的 SMT 后端。可以同时启用多个引擎，SBY 会并行运行，谁先出结果就用谁——这就是“多引擎竞赛”机制。

[script] 这是 Yosys 的命令脚本。

- `read_verilog -formal`: 读入源码并启用形式化原语支持；
- `-D FORMAL`: 定义宏，激活 `fifo_formal.v` 中的 ``ifdef FORMAL` 块；
- `hierarchy`: 展开模块层次；
- `prep`: 形式化专用的预处理流程；
- `chformal -assume -early`: 把 `assume` 提前到综合早期阶段，有助于提升求解效率。

[files] 列出所有需要参与编译的源文件，路径相对于 `.sby` 文件所在目录。

提示

`mode prove` 与 `mode bmc` 的选择策略：开发初期用 `bmc + depth 20` 快速跑通流程、抓明显 Bug；待设计稳定后再切换到 `prove` 做完整证明。两者可以通过 SBY 的 `[tasks]` 段并行运行，详见 SBY 官方文档。

4.5 Step 4: 执行验证

在 `.sby` 文件所在目录下执行：

```
1 sby fifo.sby
```

Listing 7: 运行形式化验证

SBY 会在当前目录下创建一个 `fifo/` 子目录（与 `.sby` 同名），其中包含本次运行的所有中间产物：综合后的网表、求解器日志、反例波形等。运行结束后，终端会打印一段汇总信息，下一节将详细解读。

说明

若希望 SBY 在结束后自动打开 GTKWave 查看反例，可在命令后追加 `--wave: sby fifo.sby --wave`。该选项仅在验证失败时生效。

5 实验一：结果分析与调试

5.1 场景一：验证通过 (PASS)

如果设计正确、属性编写得当，SBY 的输出大致如下（关键行已加注释）：

```
1 SBY 21:43:12 [fifo] SymbiYosys (c) 2016-2024 Claire Wolf and YosysHQ
2 SBY 21:43:12 [fifo] Starting (working directory: fifo/src)
3 SBY 21:43:12 [fifo] Running script...
4 SBY 21:43:15 [fifo] PREP DONE
5 SBY 21:43:15 [fifo] Engine smtbmc_z3: starting
6 SBY 21:43:18 [fifo] Engine smtbmc_z3: Reached depth 20
7 SBY 21:43:19 [fifo] Engine smtbmc_z3: Proved all assertions
8 SBY 21:43:19 [fifo] Engine smtbmc_z3: finished (returncode=0)
9 SBY 21:43:19 [fifo] DONE
10 SBY 21:43:19 [fifo] summary: Elapsed clock time [h:mm:ss]: 0:00:07
11 SBY 21:43:19 [fifo] summary: engine_0 (smtbmc z3) returned PASS
12 SBY 21:43:19 [fifo] summary: successful proof, all assertions passed.
13 SBY 21:43:19 [fifo] DONE
```

Listing 8: 验证通过时的典型日志

关键判读点：

- PREP DONE: Yosys 综合阶段完成，网表已生成；
- Reached depth 20: 求解器已展开到第 20 拍；
- Proved all assertions: 求解器成功证明了所有 assert 在 depth 20 范围内恒成立；
- returned PASS: 引擎返回”通过”状态；
- successful proof, all assertions passed: SBY 汇总结论——验证成功。

此时 fifo/ 目录下不会生成 engine_0.vcd 文件（因为没有反例），但会保留 logfile.txt 供事后审计。

5.2 场景二：验证失败 (Counter-example)

由于我们在 Step 1 故意埋设了 full 逻辑的 Bug，实际运行本实验时**必然失败**。SBY 的输出会变成：

```
1 SBY 21:50:01 [fifo] SymbiYosys (c) 2016-2024 Claire Wolf and YosysHQ
2 SBY 21:50:01 [fifo] Starting (working directory: fifo/src)
3 SBY 21:50:04 [fifo] PREP DONE
4 SBY 21:50:04 [fifo] Engine smtbmc_z3: starting
5 SBY 21:50:05 [fifo] Engine smtbmc_z3: Checking assertions in step 1..
6 SBY 21:50:05 [fifo] Engine smtbmc_z3: Checking assertions in step 2..
7 SBY 21:50:05 [fifo] Engine smtbmc_z3: Checking assertions in step 3..
8 SBY 21:50:06 [fifo] Engine smtbmc_z3: Assert failed in fifo_formal:\
```

```

9      fifo_formal.v:62: (count == DEPTH) == full
10 SBY 21:50:06 [fifo] Engine smtbmc_z3: Writing trace to VCD file:\
11      fifo/engine_0/trace.vcd
12 SBY 21:50:06 [fifo] Engine smtbmc_z3: finished (returncode=12)
13 SBY 21:50:06 [fifo] summary: Elapsed clock time [h:mm:ss]: 0:00:05
14 SBY 21:50:06 [fifo] summary: engine_0 (smtbmc z3) returned FAIL
15 SBY 21:50:06 [fifo] summary: counterexample trace: fifo/engine_0/trace.vcd
16 SBY 21:50:06 [fifo] DONE

```

Listing 9: 验证失败时的典型日志（反例已生成）

关键判读点：

- Checking assertions in step N.: 求解器逐拍展开检查，N 是拍号；
- Assert failed in fifo_formal: fifo_formal.v:62: 精确指出违例发生在 fifo_formal.v 第 62 行的断言，即我们写的”陷阱”断言 6；
- Writing trace to VCD file: fifo/engine_0/trace.vcd: 求解器已经把反例轨迹写入 VCD 文件，这是调试的核心线索；
- returncode=12: SBY 内部约定的”找到反例”返回码；
- returned FAIL 与 counterexample trace: 明确告知本次验证失败，并给出反例文件路径。

5.3 利用反例波形定位逻辑错误

拿到 trace.vcd 后，用 GTKWave 打开：

```

1 gtkwave fifo/engine_0/trace.vcd &

```

Listing 10: 用 GTKWave 打开反例波形

在 GTKWave 中，按以下步骤分析反例：

1. **展开信号树：**左侧 SST 窗口中展开 fifo_formal → dut，把 clk、rst_n、wr_en、rd_en、wr_ptr、rd_ptr、full、empty、count 全部拖入右侧信号窗口。
2. **定位违例时刻：**SBY 通常会在波形末尾标记违例发生的拍号。本例中违例出现在第 3 拍左右——此时 count 还远未达到 DEPTH=4，但 full 已经被置 1。
3. **回溯触发条件：**观察违例前的几拍信号，可以看到 wr_en 连续有效、wr_ptr 从 00 → 01 → 10 → 11，当 wr_ptr 回到 00（即第 4 次写入时低 2 位再次为 0），full 误判为 1，而此时 count 仅为 2。
4. **对照设计代码：**回到 fifo.v 第 41 行，确认 full 的比较条件确实只用了低 PTR_W 位，与波形现象完全吻合。

修复方案：将 `full` 的判断改为标准的”最高位不同、低位相等”：

```
1 // 正确写法：最高位不同、低位相等时为满
2 assign full = (wr_ptr[PTR_W] != rd_ptr[PTR_W]) &&
3           (wr_ptr[PTR_W-1:0] == rd_ptr[PTR_W-1:0]);
```

Listing 11: 修复后的 `full` 逻辑

修改后重新运行 `sby fifo.sby`，本次应能顺利通过。如果仍然失败，说明属性本身或其它逻辑还有问题，需要继续按上述流程迭代。

5.4 常见调试技巧汇总

表 6: 形式化验证常见问题与排查思路

现象	排查思路
求解器长时间不返回	1) 降低 <code>depth</code> ; 2) 检查 <code>assume</code> 是否过松导致状态爆炸; 3) 切换更快的引擎 (如 <code>boolector</code>)
大量”伪反例” <code>cover</code> 失败	<code>assume</code> 不足, 外部输入空间未约束; 补充复位、握手、范围约束 验证深度不够或约束过严; 提高 <code>depth</code> 或放宽 <code>assume</code>
<code>Assert failed</code> 在第 1 拍	通常是初始状态未约束, 检查 <code>initial</code> 中的 <code>assume</code>
<code>Assert failed</code> 但波形看似正常	检查 <code>\$past</code> 在第 0 拍的返回值 (默认为 X), 可能需要 <code>\$initstate</code> 守卫

提示

形式化验证是一个”迭代收敛”的过程: 写属性 → 跑 SBY → 看反例 → 修设计或修属性 → 再跑。不要期望一次就能跑通。每次失败都是一次学习机会, 反例波形就是最好的”老师”。

5.5 本实验小结

通过本实验, 我们完整走过了”编写 DUT → 编写属性 → 配置 SBY → 执行验证 → 分析反例 → 修复缺陷”的全流程。关键收获包括:

1. 形式化验证与仿真互补, 前者覆盖完整状态空间, 后者覆盖具体场景, 二者结合才能构建高置信度的验证体系;
2. `assume` / `assert` / `cover` 三类原语各司其职, 混淆会导致”假通过”或”伪反例”;
3. SBY 的 `.sby` 配置文件结构清晰, `[options]`、`[engines]`、`[script]` 三段是核心;
4. 反例波形是定位 Bug 的金钥匙, 学会读波形比学会写属性更重要;
5. 影子计数器、包装模块等技巧能让属性代码与设计代码解耦, 便于维护。

6 实验二：AXI-Lite 总线协议验证

本实验是手册的进阶部分。我们将验证一个简化的 AXI-Lite Slave 是否符合 ARM AMBA 协议规范——这是形式化验证在工业界最典型的应用场景：**协议合规性检查**。

6.1 为什么选 AXI-Lite ?

AXI (Advanced eXtensible Interface) 是 ARM AMBA 总线家族的核心协议，几乎所有现代 SoC 都基于它互联。AXI-Lite 是 AXI 的简化子集，用于寄存器级访问，特点是：

- 5 个独立通道：读地址 (AR)、读数据 (R)、写地址 (AW)、写数据 (W)、写响应 (B)；
- 每个通道都有 VALID/READY 握手；
- VALID 一旦拉高，在 READY 到来前不能撤销；
- READY 可以依赖 VALID (组合依赖)，但 VALID 不能依赖 READY。

思考一下

AXI 协议的这些规则看似简单，但用仿真测试覆盖率极低——你很难构造出所有可能的 VALID/READY 时序组合。而形式化验证可以自动搜索所有合规的握手序列，证明 Slave 在任何情况下都不会违反协议。这正是形式化验证的”主场”。

6.2 AXI-Lite 协议要点回顾

在写属性前，必须先把协议规则想清楚。下面列出本实验要验证的核心规则(来自 ARM AMBA AXI and ACE Protocol Specification)：

表 7: AXI-Lite 协议核心规则（本实验验证项）

编号	规则	验证方式
R1	VALID 一旦拉高，在 READY 拉高前不能下降	assert
R2	READY 允许依赖 VALID，反之禁止	assume (Master 侧)
R3	写事务必须有 B 通道响应	assert
R4	B 响应仅在 AW+W 都完成后才能产生	assert
R5	读数据必须与读地址一一对应（不能丢、不能多）	assert
R6	复位时所有 VALID 必须为 0	assert
R7	VALID 不能依赖 READY（组合环路禁止）	assert

通俗理解

通俗理解 AXI 握手：把 VALID 想象成”我端着菜站在窗口”，把 READY 想象成”厨师说’放这儿’”。规则 R1 说：”菜一旦端起来，厨师没说’放这儿’之前不能撤回”——否则厨师转身拿盘子时菜就掉了。规则 R7 说：”你不能等厨师说’放这儿’才决定端不端菜”——否则两人互相等，死锁。

6.3 Step 1: 编写 AXI-Lite Slave DUT

下面是一个简化的 AXI-Lite Slave，内部有 4 个 32 位寄存器，支持读写访问。我们 **故意**在 AWVALID 的处理上埋一个 Bug——它违反了规则 R1 (VALID 中途撤销)，形式化验证会立刻抓出来。

```
1 // 简化的 AXI-Lite Slave: 4 个 32 位寄存器
2 // 故意埋设 Bug: bvalid 在某些情况下会被错误撤销 (违反协议 R1)
3 module axilite_slave #(
4     parameter ADDR_W = 4,          // 4 位地址, 寻址 16 字节 = 4 个 32
        位寄存器
5     parameter DATA_W = 32
6 )(
7     input  wire          clk,
8     input  wire          rst_n,
9
10    // 写地址通道 AW
11    input  wire [ADDR_W-1:0] awaddr,
12    input  wire          awvalid,
13    output wire          awready,
14
15    // 写数据通道 W
16    input  wire [DATA_W-1:0] wdata,
17    input  wire          wvalid,
18    output wire          wready,
19
20    // 写响应通道 B
21    output wire [1:0]        bresp,    // 00=OKAY, 10=SLVERR,
        11=DECERR
22    output wire          bvalid,
23    input  wire          bready,
24
25    // 读地址通道 AR
26    input  wire [ADDR_W-1:0] araddr,
27    input  wire          arvalid,
28    output wire          arready,
29
30    // 读数据通道 R
31    output wire [DATA_W-1:0] rdata,
32    output wire [1:0]        rresp,
33    output wire          rvalid,
34    input  wire          rready
35 );
36 // 内部寄存器阵列
37 reg [DATA_W-1:0] regs [0:3];
38
```

```

39 // 写地址 / 写数据握手状态
40 reg          aw_done;    // AW 通道已完成
41 reg          w_done;    // W 通道已完成
42 reg [ADDR_W-1:0] aw_addr_latched;
43
44 // AW 通道：一旦 awvalid and awready, 锁存地址
45 assign awready = !aw_done;    // 简单握手：未完成时随时可接收
46 always @(posedge clk or negedge rst_n) begin
47     if (!rst_n) begin
48         aw_done <= 1'b0;
49         aw_addr_latched <= 0;
50     end else if (awvalid && awready) begin
51         aw_done <= 1'b1;
52         aw_addr_latched <= awaddr;
53     end else if (bvalid && bready) begin
54         // 事务完成后清零，准备接收下一次
55         aw_done <= 1'b0;
56     end
57 end
58
59 // W 通道
60 assign wready = !w_done;
61 reg [DATA_W-1:0] w_data_latched;
62 always @(posedge clk or negedge rst_n) begin
63     if (!rst_n) begin
64         w_done <= 1'b0;
65         w_data_latched <= 0;
66     end else if (wvalid && wready) begin
67         w_done <= 1'b1;
68         w_data_latched <= wdata;
69     end else if (bvalid && bready) begin
70         w_done <= 1'b0;
71     end
72 end
73
74 // B 通道：当 AW 和 W 都完成时，产生 bvalid
75 // BUG: 当 bvalid 已拉高但 bready 未到时，下一拍若 aw_done
76 //      被清零（不会，但
77 //      假设这里有逻辑错误），bvalid 会被撤销 —— 违反 R1
78 reg bvalid_r;
79 always @(posedge clk or negedge rst_n) begin
80     if (!rst_n) begin
81         bvalid_r <= 1'b0;
82     end else if (aw_done && w_done && !bvalid_r) begin
83         bvalid_r <= 1'b1;
84     end else if (bvalid_r && bready) begin

```

```

84         bvalid_r <= 1'b0;
85     end
86     // BUG 注入：当 aw_done 与 w_done
      同时被清零（理论上不会，但形式化会探索到）
87     // 时 bvalid 也被清零 —— 这里我们故意写错条件
88     else if (!aw_done && !w_done) begin
89         bvalid_r <= 1'b0;    // BUG: 不应在 bready 未到时撤销 bvalid
90     end
91 end
92 assign bvalid = bvalid_r;
93 assign bresp  = 2'b00;      // 永远返回 OKAY
94
95 // 实际写入寄存器：在 bvalid 拉起的那一拍写入
96 always @(posedge clk or negedge rst_n) begin
97     if (!rst_n) begin
98         regs[0] <= 32'h0;
99         regs[1] <= 32'h0;
100        regs[2] <= 32'h0;
101        regs[3] <= 32'h0;
102    end else if (aw_done && w_done && !bvalid_r) begin
103        case (aw_addr_latched[ADDR_W-1:2])
104            2'd0: regs[0] <= w_data_latched;
105            2'd1: regs[1] <= w_data_latched;
106            2'd2: regs[2] <= w_data_latched;
107            2'd3: regs[3] <= w_data_latched;
108        endcase
109    end
110 end
111
112 // AR 通道
113 reg                ar_done;
114 reg [ADDR_W-1:0]   ar_addr_latched;
115 assign arready = !ar_done;
116 always @(posedge clk or negedge rst_n) begin
117     if (!rst_n) begin
118         ar_done <= 1'b0;
119         ar_addr_latched <= 0;
120     end else if (arvalid && arready) begin
121         ar_done <= 1'b1;
122         ar_addr_latched <= araddr;
123     end else if (rvalid && rready) begin
124         ar_done <= 1'b0;
125     end
126 end
127
128 // R 通道

```

```

129     reg                rvalid_r;
130     reg [DATA_W-1:0]   rdata_r;
131     always @(posedge clk or negedge rst_n) begin
132         if (!rst_n) begin
133             rvalid_r <= 1'b0;
134             rdata_r  <= 32'h0;
135         end else if (ar_done && !rvalid_r) begin
136             rvalid_r <= 1'b1;
137             case (ar_addr_latched[ADDR_W-1:2])
138                 2'd0: rdata_r <= regs[0];
139                 2'd1: rdata_r <= regs[1];
140                 2'd2: rdata_r <= regs[2];
141                 2'd3: rdata_r <= regs[3];
142             endcase
143         end else if (rvalid_r && rready) begin
144             rvalid_r <= 1'b0;
145         end
146     end
147     assign rvalid = rvalid_r;
148     assign rdata  = rdata_r;
149     assign rresp  = 2'b00;
150 endmodule

```

Listing 12: 文件名: axilite_slave.v ——待测的 AXI-Lite Slave (含 Bug)

6.3.1 Bug 在哪里？

请看 bvalid_r 的最后一段 else if:

```

1 else if (!aw_done && !w_done) begin
2     bvalid_r <= 1'b0;    // BUG: 不应在 bready 未到时撤销 bvalid
3 end

```

Listing 13: Bug 代码片段

这段逻辑的”本意”可能是想处理某种边界情况，但它违反了 AXI 协议 R1: BVALID 一旦拉高，在 BREADY 到来前不能下降。当 aw_done 和 w_done 因为某种原因被清零（例如形式化求解器构造的边界输入），bvalid 会被错误撤销。

提示

这个 Bug 在仿真中极难发现——需要构造”BVALID 拉高后、BREADY 未到、同时 AW/W 状态被清零”的特定时序。而形式化验证会自动搜索所有可能的输入组合，瞬间定位。

6.4 Step 2: 编写协议形式化属性

AXI 协议验证的核心是把协议文档的文字规则翻译成 `assume/assert`。我们新建 `axilite_formal.v`, 作为 Master 侧的”形式化激励发生器”——它约束 Master 行为合规 (`assume`), 并验证 Slave 响应合规 (`assert`)。

```
1 `ifdef FORMAL
2 module axilite_formal;
3     localparam ADDR_W = 4;
4     localparam DATA_W = 32;
5
6     reg                clk = 0;
7     reg                rst_n = 0;
8
9     // Master 驱动的信号 (作为 DUT 的输入)
10    reg [ADDR_W-1:0]    awaddr, araddr;
11    reg                awvalid, wvalid, bready, arvalid, rready;
12    reg [DATA_W-1:0]    wdata;
13
14    // DUT 输出
15    wire                aready, wready, bvalid, arready, rvalid;
16    wire [1:0]          bresp, rresp;
17    wire [DATA_W-1:0]    rdata;
18
19    axilite_slave #(.ADDR_W(ADDR_W), .DATA_W(DATA_W)) dut (
20        .clk(clk), .rst_n(rst_n),
21        .awaddr(awaddr), .awvalid(awvalid), .aready(aready),
22        .wdata(wdata), .wvalid(wvalid), .wready(wready),
23        .bresp(bresp), .bvalid(bvalid), .bready(bready),
24        .araddr(araddr), .arvalid(arvalid), .arready(arready),
25        .rdata(rdata), .rresp(rresp), .rvalid(rvalid), .rready(rready)
26    );
27
28    // ----- 复位约束 -----
29    initial begin
30        assume(!rst_n);
31        assume(awvalid == 1'b0);
32        assume(wvalid == 1'b0);
33        assume(arvalid == 1'b0);
34        assume(bready == 1'b0);
35        assume(rready == 1'b0);
36    end
37    always @(posedge clk) begin
38        if ($past(rst_n)) assume(rst_n); // 复位释放后不再返回
39    end
40
```

```

41 // ----- 规则 R1: VALID 一旦拉高, READY 到来前不能下降 -----
42 // 这是 Master 侧的承诺 (assume), 也是 Slave 侧的承诺 (assert)
43 // 这里我们约束 Master 行为合规
44 always @(posedge clk) begin
45     if (rst_n && !$initstate) begin
46         // AWVALID 不能在 AWREADY 到来前撤销
47         if ($past(awvalid) && !$past(awready))
48             assume(awvalid);
49         // WVALID
50         if ($past(wvalid) && !$past(wready))
51             assume(wvalid);
52         // ARVALID
53         if ($past(arvalid) && !$past(arready))
54             assume(arvalid);
55         // BREADY 与 RREADY 可以自由变化 (Master 可以随时撤销 ready)
56     end
57 end
58
59 // ----- 规则 R6: 复位时所有 VALID 必须为 0 -----
60 always @(posedge clk) begin
61     if (!rst_n) begin
62         assert(!awvalid);
63         assert(!wvalid);
64         assert(!arvalid);
65         assert(!bvalid);
66         assert(!rvalid);
67     end
68 end
69
70 // ----- 规则 R1 (Slave 侧): BVALID/RVALID 不能在 READY
71 // 到来前撤销 -----
72 always @(posedge clk) begin
73     if (rst_n && !$initstate) begin
74         // BVALID 一旦拉高, BREADY 到来前不能下降
75         if ($past(bvalid) && !$past(bready))
76             assert(bvalid); // 这里会触发反例!
77         // RVALID 同理
78         if ($past(rvalid) && !$past(rready))
79             assert(rvalid);
80     end
81 end
82
83 // ----- 规则 R3/R4: B 响应仅在 AW+W 完成后产生 -----
84 // 用影子状态机跟踪 AW/W/B 的事务生命周期
85 reg [1:0] w_state; // 0=idle, 1=aw_done, 2=w_done, 3=b_pending
86 always @(posedge clk or negedge rst_n) begin

```

```

86     if (!rst_n) begin
87         w_state <= 0;
88     end else begin
89         case (w_state)
90             0: if (awvalid && awready) w_state <= 1;
91             1: if (wvalid && wready)   w_state <= 2;
92             2: if (bvalid)             w_state <= 3;
93             3: if (bvalid && bready)   w_state <= 0;
94         endcase
95     end
96 end
97
98 always @(posedge clk) begin
99     if (rst_n) begin
100         // BVALID 只能在 w_state >= 2 时拉高
101         if (bvalid) assert(w_state >= 2);
102         // BVALID 拉高后必须等到 bready 才能下降 (已在 R1 中覆盖)
103     end
104 end
105
106 // ----- 规则 R5: 读数据与读地址一一对应 -----
107 reg [1:0] r_state; // 0=idle, 1=ar_done, 2=r_pending
108 always @(posedge clk or negedge rst_n) begin
109     if (!rst_n) begin
110         r_state <= 0;
111     end else begin
112         case (r_state)
113             0: if (arvalid && arready) r_state <= 1;
114             1: if (rvalid)             r_state <= 2;
115             2: if (rvalid && rready)   r_state <= 0;
116         endcase
117     end
118 end
119 always @(posedge clk) begin
120     if (rst_n) begin
121         if (rvalid) assert(r_state >= 1);
122     end
123 end
124
125 // ----- 覆盖目标 -----
126 always @(posedge clk) begin
127     cover(bvalid); // 至少看到一次写响应
128     cover(rvalid); // 至少看到一次读数据
129     cover(awvalid && awready); // 至少完成一次 AW 握手
130     cover(bvalid && bready);   // 至少完成一次 B 握手
131 end

```

```

132
133     // 生成时钟
134     initial begin
135         #5 clk = 1;
136         forever #5 clk = ~clk;
137     end
138 endmodule
139 `endif

```

Listing 14: 文件名: axilite_formal.v ——AXI-Lite 协议属性

6.4.1 属性设计思路逐条解析

复位约束 与 FIFO 实验同理，强制第 0 拍复位且所有 VALID=0。这是 AXI 协议 R6 的要求——复位时 Master 不能发起任何事务。

规则 R1 (Master 侧 assume) 我们用 `assume` 约束 Master 行为合规: AWVALID 一旦拉高，在 AWREADY 到来前不能下降。这相当于告诉求解器: ”只考虑合规的 Master 激励”——否则求解器会构造各种违规激励，产生大量与 Slave 设计无关的反例。

关键技巧: 用 `$past` 检查”上一拍 VALID 为 1 且 READY 为 0”的情况，本拍 VALID 必须仍为 1。

规则 R1 (Slave 侧 assert) 同样的规则，对 Slave 的 BVALID 和 RVALID 用 `assert` 验证。这是我们要检查的核心——Slave 不能违反协议。本实验的 Bug 正是违反了这条规则。

为什么 Master 用 `assume`、Slave 用 `assert`? 因为我们把 Master 当作”合规的外部环境”(只验证 Slave)，所以约束它；Slave 是被验证对象，所以断言它。

影子状态机 (w_state / r_state) 与 FIFO 实验的影子计数器同理——用一个独立的小状态机跟踪事务生命周期,再用 `assert` 把它与 DUT 的输出绑定。例如 `if (bvalid) assert(w_state >= 2)` 确保 BVALID 只在 AW+W 都完成后才出现。

覆盖目标 确保关键握手场景都被探索到。如果 `cover(bvalid && bready)` 失败，说明求解器从未完成一次完整的写事务——可能是约束过严，验证本身不够充分。

注意

AXI 验证中最容易犯的错误是把 **Master 的合规约束写成 `assert`**。例如 `assert(!($past(awvalid) && !$past(awready) && !awvalid))`——这会要求求解器证明 Master 合规，但 Master 是自由输入，求解器会立刻构造一个违规激励让断言失败，产生”伪反例”。正确做法是用 `assume` 约束 Master。

6.5 Step 3: 编写 SBY 脚本


```

1 [options]
2 mode bmc                # AXI 协议验证推荐先用 bmc 快速抓 Bug
3 depth 30                # AXI 事务需要多拍，深度适当加大
4 multiclock off
5 wait 0
6
7 [engines]
8 smtbmc z3               # 主引擎
9 # smtbmc boolelector    # 备选
10
11 [script]
12 read_verilog -formal -D FORMAL axilite_slave.v axilite_formal.v
13 hierarchy -top axilite_formal
14 prep -top axilite_formal
15 chformal -assume -early
16
17 [files]
18 axilite_slave.v
19 axilite_formal.v

```

Listing 15: 文件名：axilite.sby ——AXI-Lite 验证配置

6.5.1 与 FIFO 实验配置的差异

- mode bmc 而非 prove: AXI 协议涉及多通道、多拍事务，prove 模式可能长时间不收敛。先用 bmc 抓 Bug，待设计稳定再考虑 prove；
- depth 30: AXI 一次完整写事务需要 4-6 拍（AW/W/B 各 1-2 拍），深度太小无法覆盖完整事务；
- 其余结构与 FIFO 实验一致。

6.6 Step 4: 执行验证

```

1 sby axilite.sby

```

Listing 16: 运行 AXI-Lite 验证

6.7 结果分析：协议违例反例

由于我们故意埋设了 bvalid 撤销的 Bug，验证必然失败。典型日志如下：

```

1 SBY 22:10:33 [axilite] SymbiYosys (c) 2016-2024 Claire Wolf and YosysHQ
2 SBY 22:10:33 [axilite] Starting (working directory: axilite/src)
3 SBY 22:10:36 [axilite] PREP DONE

```

```

4 SBY 22:10:36 [axilite] Engine smtbmc_z3: starting
5 SBY 22:10:37 [axilite] Engine smtbmc_z3: Checking assertions in step 1..
6 ...
7 SBY 22:10:41 [axilite] Engine smtbmc_z3: Checking assertions in step 7..
8 SBY 22:10:42 [axilite] Engine smtbmc_z3: Assert failed in axilite_formal:\
9     axilite_formal.v:88: ($past(bvalid) && !$past(bready)) -> bvalid
10 SBY 22:10:42 [axilite] Engine smtbmc_z3: Writing trace to VCD file:\
11     axilite/engine_0/trace.vcd
12 SBY 22:10:42 [axilite] summary: engine_0 (smtbmc z3) returned FAIL
13 SBY 22:10:42 [axilite] summary: counterexample trace:
14     axilite/engine_0/trace.vcd
14 SBY 22:10:42 [axilite] DONE

```

Listing 17: AXI-Lite 验证失败日志

关键判读点:

- 违例发生在第7拍,位置是 `axilite_formal.v:88`——即我们写的”BVALID 不能在 BREADY 到来前撤销”的断言;
- 反例轨迹已写入 `trace.vcd`。

6.7.1 用 GTKWave 分析反例

```

1 gtkwave axilite/engine_0/trace.vcd &

```

Listing 18: 打开 AXI 反例波形

在 GTKWave 中重点关注以下信号,按时间顺序观察:

1. **第 1–2 拍**: 复位阶段,所有 `VALID=0`,符合 R6;
2. **第 3 拍**: `AWVALID` 拉高, `AWREADY` 也拉高, `AW` 握手完成, `aw_done` 置 1;
3. **第 4 拍**: `WVALID` 拉高, `WREADY` 也拉高, `W` 握手完成, `w_done` 置 1;
4. **第 5 拍**: `BVALID` 拉高 (因为 `aw_done && w_done`), 但 `BREADY=0` (Master 暂时不接收响应);
5. **第 6 拍: Bug 触发**——由于某种原因 `aw_done` 和 `w_done` 同时被清零 (可能是求解器构造的边界输入), `bvalid_r` 进入错误的 `else if` 分支, 被清零;
6. **第 7 拍**: 断言触发——`$past(bvalid)=1` 且 `$past(bready)=0`, 但本拍 `bvalid=0`, 违反 R1。

6.7.2 修复方案

删除 bvalid_r 逻辑中错误的 else if 分支：

```
1 reg bvalid_r;
2 always @(posedge clk or negedge rst_n) begin
3     if (!rst_n) begin
4         bvalid_r <= 1'b0;
5     end else if (aw_done && w_done && !bvalid_r) begin
6         bvalid_r <= 1'b1;
7     end else if (bvalid_r && bready) begin
8         bvalid_r <= 1'b0;
9     end
10    // 删除了错误的 else if (!aw_done && !w_done) 分支
11 end
```

Listing 19: 修复后的 bvalid 逻辑

修复后重新运行 sby axilite.sby，所有断言应能通过。

6.8 AXI-Lite 验证进阶技巧

6.8.1 多引擎竞赛

对于复杂的 AXI 设计，建议同时启用多个引擎，让它们“赛跑”：

```
1 [engines]
2 smtbmc z3
3 smtbmc boolector
4 aigbmc
```

Listing 20: 多引擎配置

SBY 会并行启动三个引擎，谁先返回结果（PASS 或 FAIL）就用谁的结果。这能显著缩短验证时间——不同引擎对不同类型的问题有各自的强项。

6.8.2 分阶段验证

不要一开始就跑全部断言。建议分阶段：

1. **第一阶段**：只验证复位规则 R6，确保初始状态正确；
2. **第二阶段**：加入 R1（VALID 稳定性），抓最基础的协议违例；
3. **第三阶段**：加入 R3/R4/R5（事务完整性），验证功能正确性；
4. **第四阶段**：切换到 prove 模式，做完整证明。

每阶段通过后再进入下一阶段，能快速定位问题层次。

6.8.3 用 [tasks] 并行多模式

SBY 支持用 [tasks] 段同时跑 bmc 和 prove:

```
1 [tasks]
2 bmc      # 任务 1: 有界模型检测
3 prove    # 任务 2: 完整证明
4
5 [options]
6 bmc: mode bmc
7 bmc: depth 30
8 prove: mode prove
9 prove: depth 30
10
11 [engines]
12 smtbmc z3
13
14 [script]
15 read_verilog -formal -D FORMAL axilite_slave.v axilite_formal.v
16 hierarchy -top axilite_formal
17 prep -top axilite_formal
18
19 [files]
20 axilite_slave.v
21 axilite_formal.v
```

Listing 21: tasks 并行配置

运行 `sby axilite.sby` 后, SBY 会在 `axilite_bmc/` 和 `axilite_prove/` 两个子目录下分别产出结果。

6.9 本实验小结

通过 AXI-Lite 验证实验, 我们掌握了形式化验证在**协议合规性检查**中的应用:

1. 协议验证的核心是**把文档规则翻译成 assume/assert**——文档中“Master 必须...”对应 `assume`, “Slave 必须...”对应 `assert`;
2. VALID/READY 握手规则用 `$past` 检查“上一拍”状态, 是 AXI 验证的通用模式;
3. 影子状态机能跟踪事务生命周期, 验证多拍事务的完整性;
4. 多引擎竞赛、分阶段验证、[tasks] 并行是提升效率的三大技巧;
5. 协议验证的 Bug 往往隐藏在边界时序中, 仿真极难覆盖, 形式化验证是“杀手锏”。

结语

本手册通过 FIFO 与 AXI-Lite 两个递进实验，完整展示了 Yosys + SymbiYosys 形式化验证工具链的实战用法。形式化验证并非“银弹”——它无法替代仿真，也无法解决所有验证问题。但在控制逻辑、协议合规、边界 Bug 这些“仿真盲区”，它展现出了无可替代的价值。

建议读者在掌握本手册内容后，尝试将形式化验证引入自己的实际项目——从一个小模块开始，逐步扩展到整个子系统。随着经验积累，你会发现形式化验证不仅能发现 Bug，更能倒逼你把设计的“不变式”思考清楚，从而写出更健壮的 RTL 代码。

形式化验证的学习曲线确实比仿真陡峭，但一旦跨过门槛，它将成为你工具箱中最锋利的那把刀。祝你在形式化验证的道路上越走越远。